Sistemas Operacionais (SO)

Aula 02

Prof.: Nathan Saraiva

Roteiro

- **Tipos de Sistemas Operacionais**
- **Conceitos de *Hardware* e *Software***
  - *Hardware*;
  - *Software*.

Tipos de sistemas operacionais

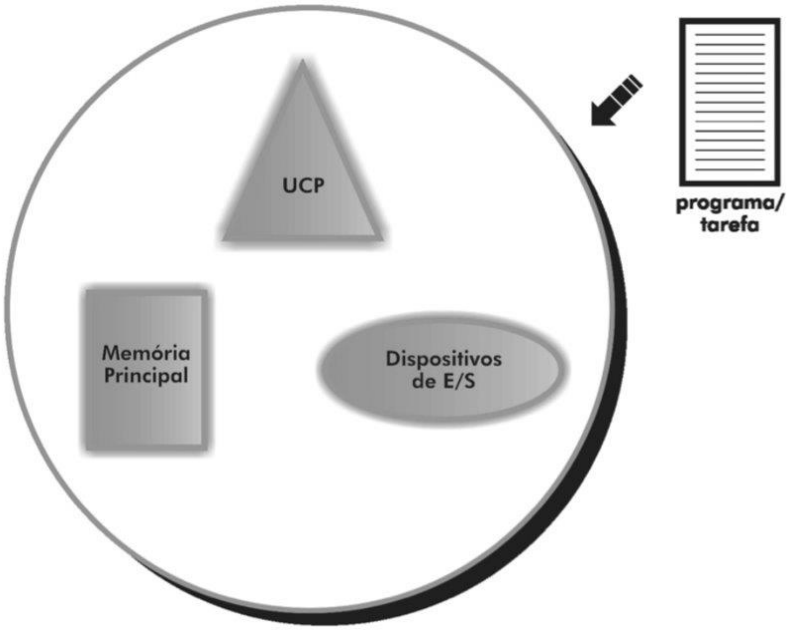
- **Visão Geral**
  - Os tipos de sistemas operacionais e sua evolução estão relacionados diretamente com a evolução do *hardware* e das aplicações por ele suportadas.

Tipos de sistemas operacionais

- **Sistemas Monoprogramáveis/Monotarefa**
  - Os primeiros sistemas operacionais eram tipicamente voltados para a execução de um único programa. Qualquer outra aplicação, para ser executada, deveria aguardar o término do programa corrente.
  - Os sistemas monoprogramáveis, se caracterizam por permitir que o processador, a memória e os periféricos permaneçam exclusivamente dedicados à execução de um **único programa**.
  - Os sistemas monoprogramáveis estão tipicamente relacionados ao surgimento dos primeiros computadores na década de 1960.

Tipos de sistemas operacionais

- **Sistemas Monoprogramáveis/Monotarefa**
  - Neste tipo de sistema, enquanto um programa aguarda por um evento, como a digitação de um dado, o processador permanece ocioso, sem realizar qualquer tipo de processamento. A memória é subutilizada caso o programa não a preencha totalmente, e os periféricos, como discos e impressoras, estão dedicados a um único usuário, nem sempre utilizados de forma integral.



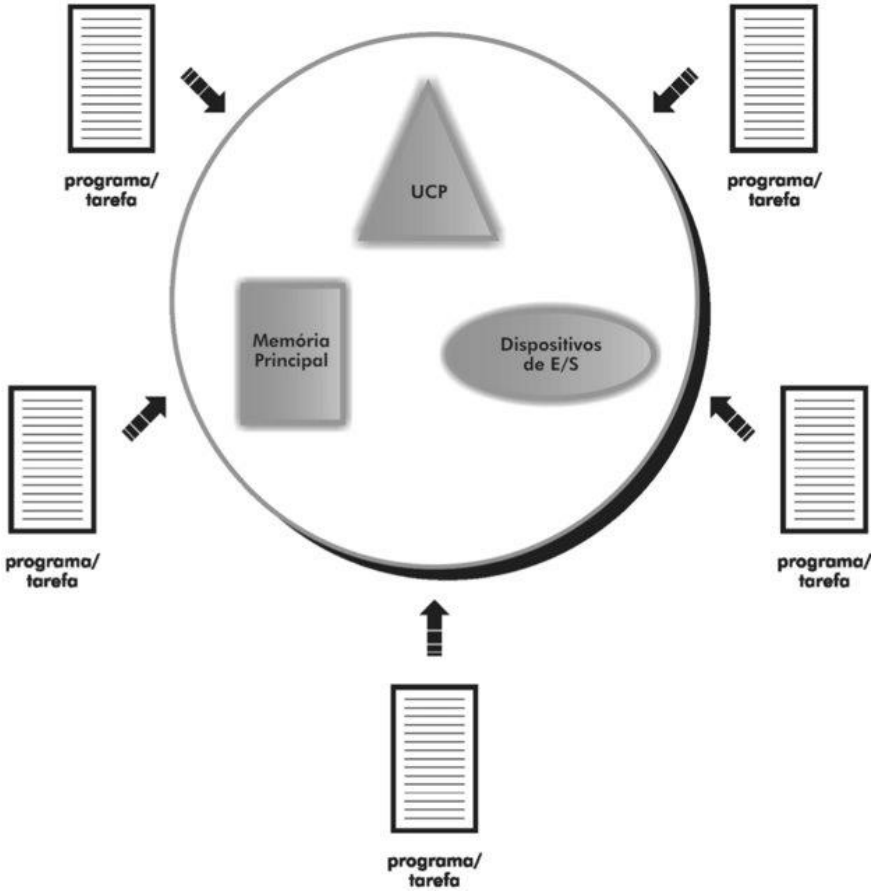
Tipos de sistemas operacionais

- **Sistemas Multiprogramáveis/Multitarefa**
  - Os sistemas multiprogramáveis ou multitarefa são uma evolução dos sistemas monoprogramáveis. Neste tipo de sistema, os recursos computacionais são compartilhados entre os diversos usuários e aplicações. Nos multiprogramáveis várias aplicações compartilham os recursos.

  - Neste tipo de sistema, por exemplo, enquanto um programa espera por uma operação de leitura ou gravação em disco, outros programas podem estar sendo processados neste mesmo intervalo de tempo. Nesse caso, podemos observar o **compartilhamento** da memória e do processador.

Tipos de sistemas operacionais

- **Sistemas Multiprogramáveis/Multitarefa**



Tipos de sistemas operacionais

- **Sistemas Multiprogramáveis/Multitarefa**
  - Vantagens:
    - Redução de custos em função da possibilidade do compartilhamento dos diversos recursos entre as diferentes aplicações.
    - Possibilitam na média a redução total do tempo de execução das aplicações.
  - Os sistemas multiprogramáveis, apesar de mais eficientes que os monoprogramáveis, são de implementação muito mais complexa.
  - A partir do número de usuários que interagem com o sistema operacional, podemos classificar os sistemas multiprogramáveis como monousuário ou multiusuário.

Tipos de sistemas operacionais

- **Sistemas com Múltiplos Processadores**
  - Os sistemas com múltiplos processadores caracterizam-se por possuir duas ou mais CPUs(UCPs) interligadas e trabalhando em conjunto.
  - A vantagem deste tipo de sistema é permitir que vários programas sejam executados ao mesmo tempo ou que um mesmo programa seja subdividido em partes para serem executadas simultaneamente em mais de um processador.
  - Com múltiplos processadores foi possível a criação de sistemas computacionais voltados, principalmente, para processamento científico, aplicado, por exemplo, no desenvolvimento aeroespacial, prospecção de petróleo, simulações, processamento de imagens e CAD.

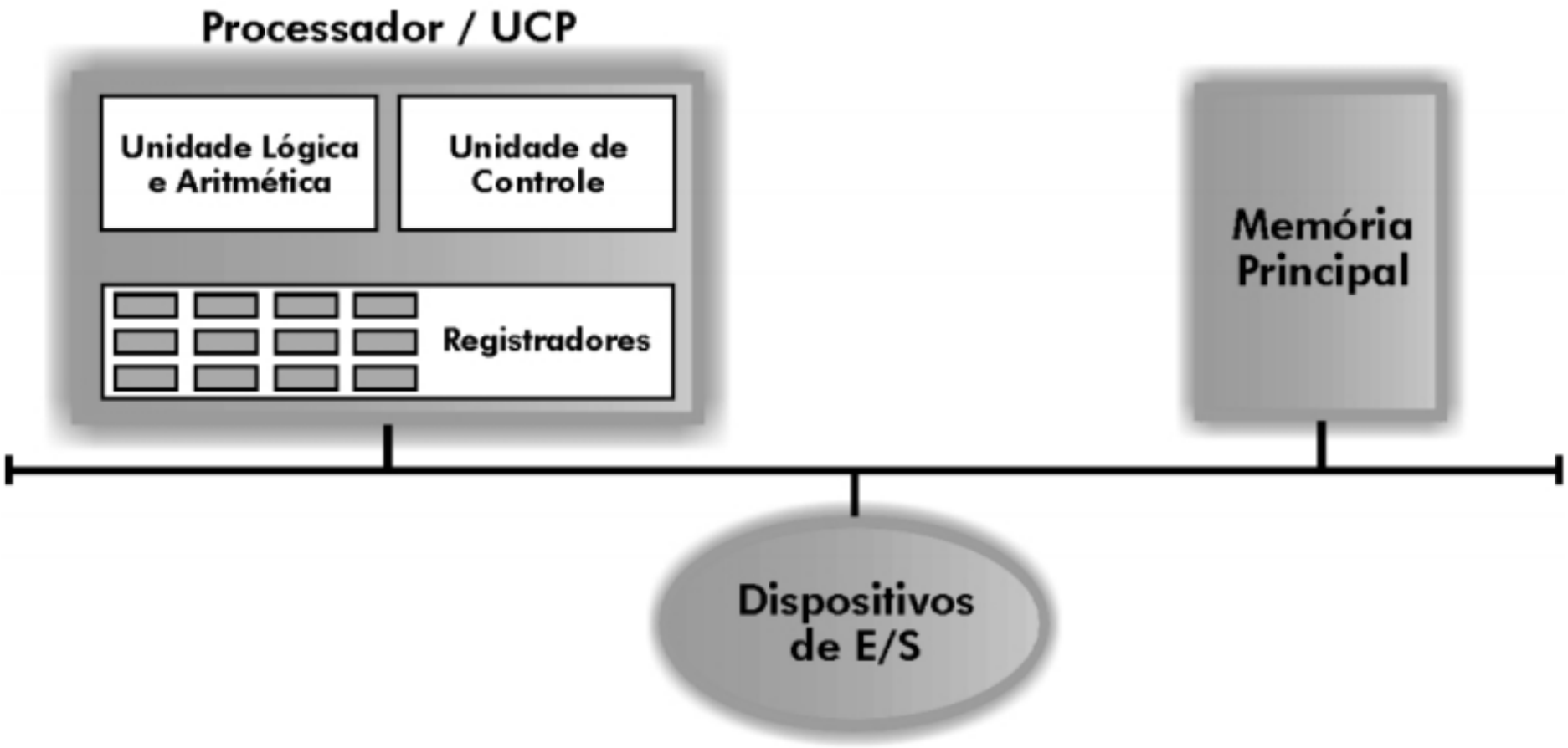
Tipos de sistemas operacionais

- **Sistemas com Múltiplos Processadores**
  - Vantagens como escalabilidade, disponibilidade e balanceamento de carga.
    - **Escalabilidade** é a capacidade de ampliar o poder computacional do sistema apenas adicionando novos processadores. Em ambientes com um único processador, caso haja problemas de desempenho, seria necessário substituir todo o sistema por uma outra configuração com maior poder de processamento. Com a possibilidade de múltiplos processadores, basta acrescentar novos processadores à configuração.
    - **Disponibilidade** é a capacidade de manter o sistema em operação mesmo em casos de falhas. Neste caso, se um dos processadores falhar, os demais podem assumir suas funções de maneira transparente aos usuários e suas aplicações, embora com menor capacidade de computação.
    - **Balanceamento de carga** é a possibilidade de distribuir o processamento entre os diversos processadores da configuração a partir da carga de trabalho de cada processador, melhorando, assim, o desempenho do sistema como um todo.

Conceitos de *Hardware* e *Software*

- ***Hardware***
  - Todos os componentes de um sistema computacional são agrupados em três subsistemas básicos, chamados unidades funcionais:
    - processador ou unidade central de processamento;
    - memória principal e;
    - dispositivos de entrada/saída.

  - Estes subsistemas estão presentes em qualquer tipo de computador digital (ver Figura 1), independentemente da arquitetura ou fabricante. Neste item serão descritos os conceitos básicos dos principais componentes desses subsistemas.


Processador / UCP
Unidade Lógica e Aritmética
Unidade de Controle
Registradores
Memória Principal
Dispositivos de E/S

Conceitos de *Hardware* e *Software*

- ***Hardware***
  - Processador
    - Também denominado unidade central de processamento (UCP), gerencia todo o sistema computacional controlando as operações realizadas por cada unidade funcional. A principal função do processador é controlar e executar instruções presentes na memória principal, por meio de operações básicas como somar, subtrair, comparar e movimentar dados.

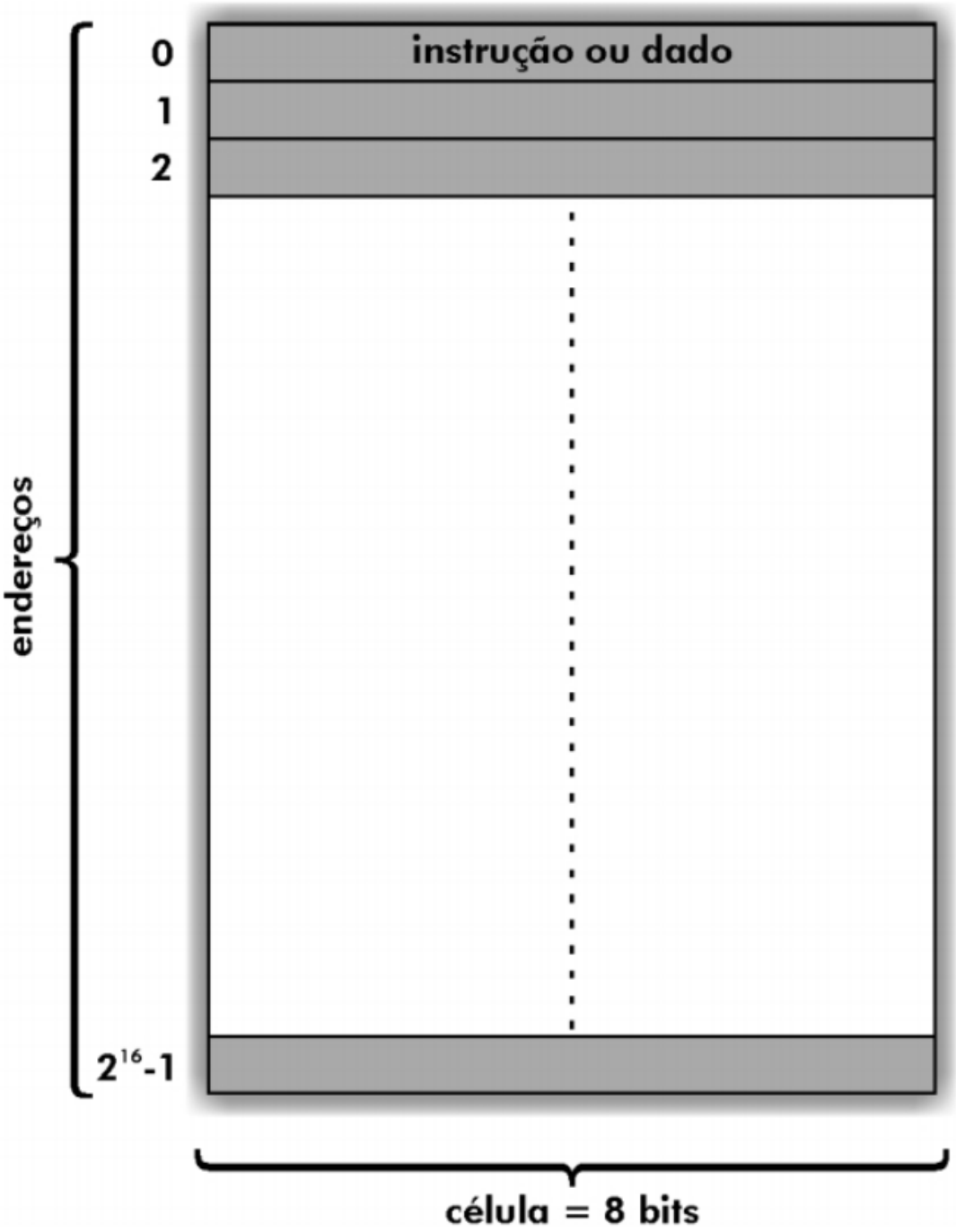
Conceitos de *Hardware* e *Software*

- ***Hardware***
  - Memória
    - A memória principal, primária ou real é o local onde são armazenados instruções e dados. A memória é composta por unidades de acesso chamadas células, sendo cada célula composta por um determinado número de *bits*. O bit é a unidade básica de memória, podendo assumir o valor lógico 0 ou 1.

    - Atualmente, a grande maioria dos computadores utiliza o *byte* (8 *bits*) como tamanho de célula, porém encontramos computadores de gerações passadas com células de 16, 32 e até mesmo 60 *bits*. Podemos concluir, então, que a memória é formada por um conjunto de células, onde cada célula possui um determinado número de *bits* (ver Figura 2).

Conceitos de *Hardware* e *Software*

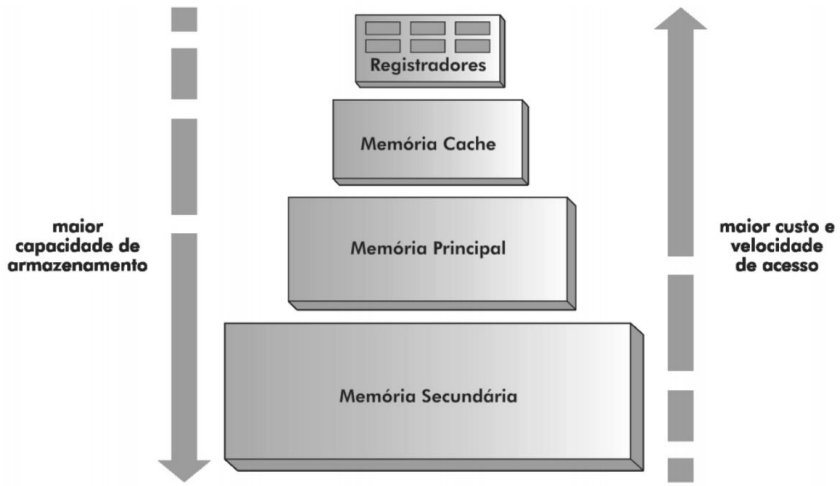
- ***Hardware***
  - <u>Memória</u>
    - O acesso ao conteúdo de uma célula é realizado através da especificação de um número chamado endereço. O endereço é uma referência única que podemos fazer a uma célula de memória. Quando um programa deseja ler ou escrever um dado em uma célula, deve primeiro especificar qual o endereço de memória desejado, para depois realizar a operação.

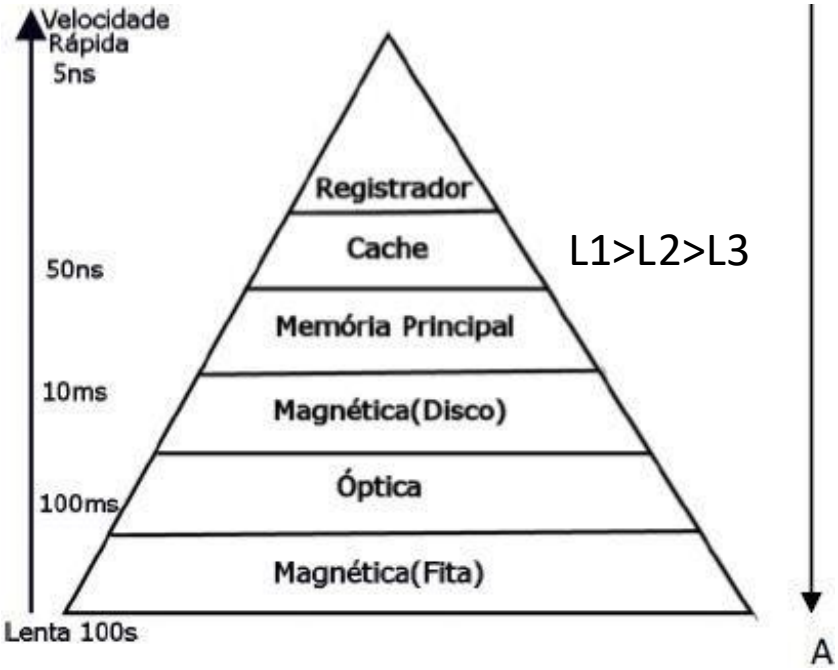

0
1
2
instrução ou dado
endereços
2^{16}-1
célula = 8 bits

Conceitos de *Hardware* e *Software*

- ***Hardware***
  - <u>Memória Secundária</u>
    - A memória secundária é um meio permanente, isto é, não volátil de armazenamento de programas e dados. Enquanto a memória principal precisa estar sempre energizada para manter suas informações, a memória secundária não precisa de alimentação.

    - O acesso à memória secundária é lento, se comparado com o acesso à memória principal, porém seu custo é baixo e sua capacidade de armazenamento é bem superior. Enquanto a unidade de acesso à memória secundária é da ordem de milissegundos, o acesso à memória principal é de nanossegundos. Podemos citar, como exemplos de memórias secundárias, a fita magnética, o disco magnético e o disco óptico.


Registradores
Memória Cache
Memória Principal
Memória Secundária
maior capacidade de armazenamento
maior custo e velocidade de acesso

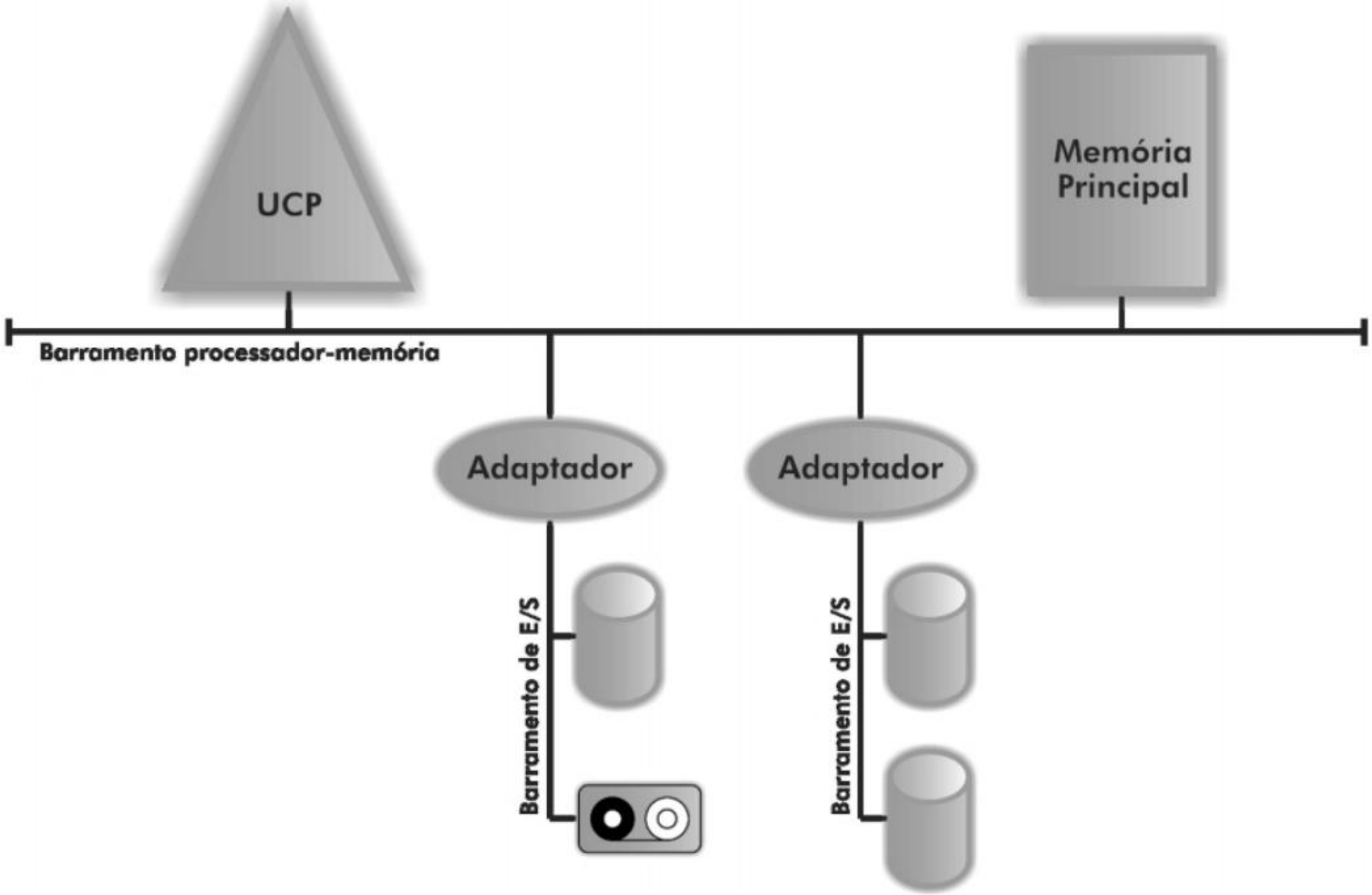

Velocidade
Rápida
5ns
50ns
10ms
100ms
Lenta 100s
Registrador
Cache
Memória Principal
Magnética(Disco)
Óptica
Magnética(Fita)
L1>L2>L3
A

Conceitos de *Hardware* e *Software*

- ***Hardware***
  - <u>Dispositivos de Entrada e Saída</u>
    - Os dispositivos de entrada e saída (E/S) são utilizados para permitir a comunicação entre o sistema computacional e o mundo externo, e podem ser divididos em duas categorias: os que são utilizados como memória secundária e os que servem para a interface usuário-máquina.

    - Os dispositivos utilizados como memória secundária (discos e fitas magnéticas) caracterizam-se por ter capacidade de armazenamento bastante superior ao da memória principal. Seu custo é relativamente baixo, porém o tempo de acesso à memória secundária é bem superior ao da memória principal.

    - Outros dispositivos têm como finalidade a comunicação usuário-máquina, como teclados, monitores de vídeo, impressoras e *plotters*. A implementação de interfaces mais amigáveis permite, cada vez mais, que usuários pouco especializados utilizem computadores de maneira intuitiva.


UCP
Memória Principal
Barramento processador-memória
Adaptador
Adaptador
Barramento de E/S
Barramento de E/S

Conceitos de *Hardware* e *Software*

- ***Software***
  - Para que o *hardware* tenha utilidade prática, deve existir um conjunto de programas utilizado como interface entre as necessidades do usuário e as capacidades do *hardware*. A utilização de *softwares* adequados às diversas tarefas e aplicações torna o trabalho dos usuários muito mais simples e eficiente.

  - No decorrer da disciplina, utilizaremos o termo utilitário sempre que desejarmos fazer referência a *softwares* relacionados mais diretamente com serviços complementares do SO, como compiladores, *linkers* e depuradores. Os *softwares* desenvolvidos pelos usuários serão denominados aplicativos ou apenas aplicações.

Conceitos de *Hardware* e *Software*

- ***Software***
  - <u>Tradutor</u>
    - Nos primeiros sistemas computacionais, o ato de programar era bastante complicado, já que o programador deveria possuir conhecimento da arquitetura da máquina e programar em painéis através de fios. Esses programas eram desenvolvidos em linguagem de máquina e carregados diretamente na memória principal para execução.

    - Com o surgimento das primeiras linguagens de montagem ou *assembly* e das linguagens de alto nível, o programador deixou de se preocupar com muitos aspectos pertinentes ao *hardware*, como em qual região da memória o programa deveria ser carregado ou quais endereços de memória seriam reservados para as variáveis. A utilização dessas linguagens facilitou a construção de programas, documentação e manutenção.

Conceitos de *Hardware* e *Software*

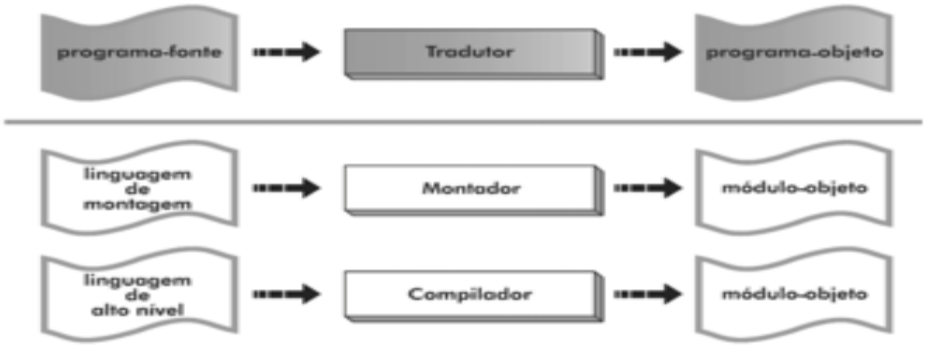
- ***Software***
  - Tradutor
    - Apesar das inúmeras vantagens proporcionadas pelas linguagens de montagem e de alto nível, os programas escritos nessas linguagens não estão prontos para ser diretamente executados pelo processador (programas-fonte). Para isso, eles têm de passar por uma etapa de conversão, onde toda representação simbólica das instruções é traduzida para código de máquina. Esta conversão é realizada por um utilitário denominado tradutor.

    - O módulo gerado pelo tradutor é denominado módulo-objeto, que, apesar de estar em código de máquina, na maioria das vezes não pode ser ainda executado.

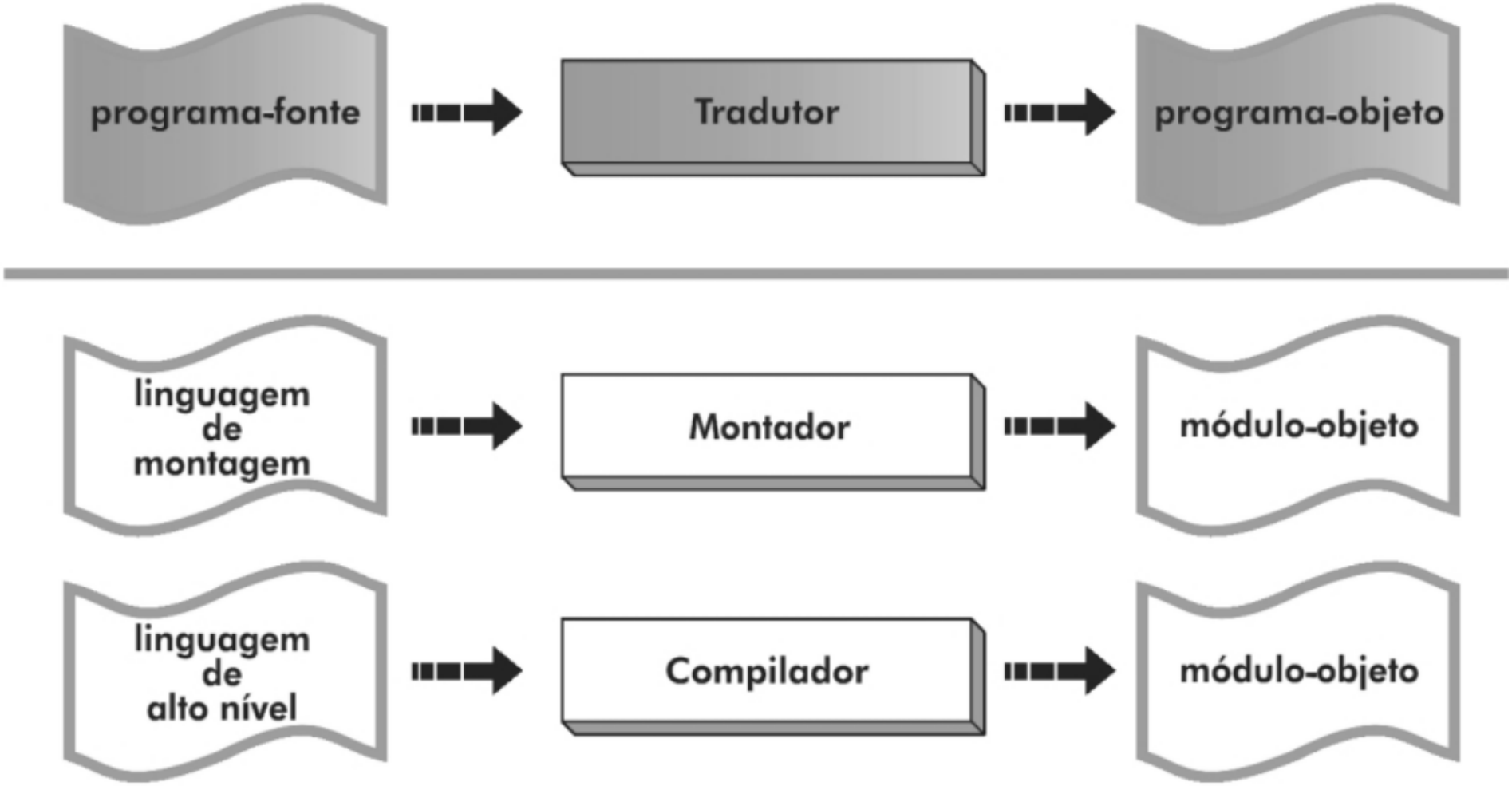
Conceitos de *Hardware* e *Software*

- ***Software***
  - <u>Tradutor</u>
    - Dependendo do tipo do programa-fonte, existem dois tipos distintos de tradutores que geram módulos-objeto: montador e compilador (ver Figura 8).

Figura 8 – Tradutor genérico.



Fonte: (MACHADO; MAIA, 2013).


programa-fonte
Tradutor
programa-objeto
linguagem de montagem
Montador
módulo-objeto
linguagem de alto nível
Compilador
módulo-objeto

Conceitos de *Hardware* e *Software*

- ***Software***
  - Tradutor
    - O montador (*assembler*) é o utilitário responsável por traduzir um programa-fonte em linguagem de montagem em um programa-objeto não executável (módulo-objeto). A linguagem de montagem é particular para cada processador, assim como a linguagem de máquina, o que não permite que programas *assembly* possam ser portados entre máquinas diferentes.

    - O compilador é o utilitário responsável por gerar, a partir de um programa escrito em uma linguagem de alto nível, um programa em linguagem de máquina não executável. As linguagens de alto nível, como Pascal, FORTRAN e COBOL, não têm nenhuma relação direta com a má- quina, ficando essa preocupação exclusivamente com o compilador.

Conceitos de *Hardware* e *Software*

- ***Software***
  - Tradutor
    - Os programadores de alto nível devem se preocupar apenas com o desenvolvimento de suas aplicações, não tendo que se envolver com detalhes sobre a arquitetura do processador. Assim, os programas-fonte podem ser portados entre computadores de diversos fabricantes, desde que existam padrões para a sintaxe da linguagem. Isso permite o desenvolvimento de aplicações independentes do equipamento.

Conceitos de *Hardware* e *Software*

- ***Software***
  - <u>Interpretador</u>
    - O interpretador é considerado um tradutor que não gera módulo-objeto. A partir de um programa-fonte escrito em linguagem de alto nível, o interpretador, durante a execução do programa, traduz cada instrução e a executa imediatamente. Algumas linguagens tipicamente interpretadas são o *Basic* e o *Perl*.

    - A maior desvantagem na utilização de interpretadores é o tempo gasto na tradução das instruções de um programa toda vez que este for executado, já que não existe a geração de um código executável. A vantagem é permitir a implementação de tipos de dados dinâmicos, ou seja, que podem mudar de tipo durante a execução do programa, aumentando, assim, sua flexibilidade.

Conceitos de *Hardware* e *Software*

- ***Software***
  - *Loader*
    - O loader é o utilitário responsável por carregar na memória principal um programa para ser executado.
  - Depurador
    - O desenvolvimento de programas está sujeito a erros de lógica, independentemente de metodologias utilizadas pelo programador. A depuração é um dos estágios desse desenvolvimento, e a utilização de ferramentas adequadas é essencial para acelerar o processo de correção dos programas. O depurador (*debugger*, em Inglês) é o utilitário que permite ao usuário acompanhar toda a execução de um programa a fim de detectar erros na sua lógica.

Conceitos de *Hardware* e *Software*

- ***Software***
  - <u>Depurador</u>
    - Este utilitário oferece ao usuário recursos como:
      - acompanhar a execução de um programa instrução por instrução;
      - possibilitar a alteração e a visualização do conteúdo de variáveis;
      - implementar pontos de parada dentro do programa (*breakpoint*, em Inglês), de forma que, durante a execução, o programa pare nesses pontos;
      - especificar que, toda vez que o conteúdo de uma variável for modificado, o programa envie uma mensagem (*watchpoint*, em Inglês).

Referências

- MACHADO, Francis Berenger; MAIA, Luiz Paulo. **Arquitetura de sistemas operacionais**. LTC, 2013.

- TANENBAUM, Andrew S.; WOODHULL, Albert S. **Sistemas Operacionais: Projetjos e Implementação**. Bookman Editora, 2009.

- SILBERSCHATZ, Abraham. **Sistemas operacionais com Java**. Elsevier Brasil, 2008.